

初识“Java虚拟线程”

北京理工大学计算机学院
金旭亮



“传统”的线程



JDK中的Thread对象，其实是对操作系统本地线程的一个封装，可称之为“平台线程（platform thread）”，在JDK 21中，为Thread类创建了一个ofPlatform()方法，用于创建一个平台线程。

//线程函数

```
Runnable task = () -> {  
    System.out.println("-----");  
    System.out.println("当前运行线程: " +  
        Thread.currentThread().getName() +  
        ", 线程id:" + Thread.currentThread().threadId());  
    System.out.println("是否背景线程? " +  
        Thread.currentThread().isDaemon());  
};
```

//普通线程，等价于:

```
// Thread.ofPlatform().unstarted(task);  
Thread thread1 = new Thread(task);  
thread1.start();  
thread1.join();  
//平台线程  
Thread thread2 = Thread.ofPlatform()  
    .daemon()  
    .unstarted(task);  
thread2.start();  
thread2.join();
```

CreatingVirtualThreads.java

虚拟线程的创建



虚拟线程可以使用`ofVirtual()`或`startVirtualThread()`方法创建.....

```
//创建一个未启动的虚拟线程
Thread thread3 = Thread.ofVirtual()
    .name("虚拟线程")
    .unstarted(task);

//启动虚拟线程
thread3.start();
thread3.join();

//创建一个虚拟线程并立即启动，未指定线程名字，名字为空。
Thread thread4 = Thread.startVirtualThread(task);
thread4.join();
```

CreatingVirtualThreads.java

```
"C:\Program Files\Java\jdk-21\bin\java.exe"

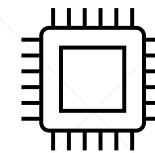
-----
当前运行线程: Thread-0,线程id:30
是否背景线程? false
-----
当前运行线程: Thread-1,线程id:31
是否背景线程? true
-----
当前运行线程: 虚拟线程,线程id:32
是否背景线程? true
-----
当前运行线程: ,线程id:37
是否背景线程? true

Process finished with exit code 0
```

平台
线程

虚拟
线程

增强的线程池功能



在JDK 21中，线程池功能也得到了增强，Executors类中添加了新的方法用于创建虚拟线程，JDK中的代码如下：

```
public static ExecutorService newVirtualThreadPoolExecutor() {  
    ThreadFactory factory = Thread.ofVirtual().factory();  
    return newThreadPoolExecutor(factory);  
}
```

这一方法完成的工作，是为每个提交给线程池的任务，都创建一个虚拟线程执行之。

虚拟线程池示例-1



```
//创建两个Map，里面放置虚拟线程和平台线程的名字
var virtualThreads = ConcurrentHashMap.<String>newKeySet();
var platformThreads = ConcurrentHashMap.<String>newKeySet();
```

//线程函数

```
Runnable task = () -> {
    //输出一个小点，表示线程正在工作
    System.out.print(".");
```

```
    Thread currentThread = Thread.currentThread();
    //将虚拟线程名字加入到Map集合中
    virtualThreads.add(currentThread.toString());
```

这种名字类似于：
runnable@ForkJoinPool-
1-worker-2

//使用正则表达式，按照名字提取出线程池中的平台线程名

```
Pattern WORKER_PATTERN = Pattern.compile("worker-[\\d?]+");
Matcher pThreadMatcher = WORKER_PATTERN.matcher(currentThread.toString());
if (pThreadMatcher.find()) {
    //抽取出负责执行的平台线程名称，加入到ConcurrentHashMap中
    //如果尝试加入同名的字符串，后来加入的会取代新加入的
    platformThreads.add(pThreadMatcher.group());
}
};
```

这种名字类似于：worker-2

UseVirtualThreadPool.java

虚拟线程池示例-2



```
//设定要执行20个工作任务
int N_TASKS = 20;
//每个工作任务, 创建一个虚拟线程
try (var executorService = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int index = 0; index < N_TASKS; index++) {
        executorService.submit(task);
    }
}
```

为每个任务创建一个
虚拟线程

向线程池提交工作任务

查看结果:

```
System.out.println("\n敲回车看结果。");
new Scanner(System.in).nextLine();
System.out.println("# 虚拟线程数:" + virtualThreads.size());
System.out.println("# 平台线程数:" + platformThreads.size());
System.out.println("\n-----虚拟线程列表-----");
virtualThreads.forEach(System.out::println);
System.out.println("\n-----平台线程列表-----");
platformThreads.forEach(System.out::println);
```

UseVirtualThreadPool.java

运行截图

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaage
.....
敲回车看结果。

# 虚拟线程数: 20
# 平台线程数:13

-----虚拟线程列表-----
VirtualThread[#32]/runnable@ForkJoinPool-1-worker-2
VirtualThread[#45]/runnable@ForkJoinPool-1-worker-9
VirtualThread[#50]/runnable@ForkJoinPool-1-worker-15
VirtualThread[#39]/runnable@ForkJoinPool-1-worker-4
```

这个实际上是负责执行此虚拟线程任务的平台线程名字

-----平台线程列表-----

worker-14
worker-9
worker-15
worker-8
worker-7
worker-2
worker-1
worker-10
worker-6
worker-11
worker-5
worker-4
worker-3

通过从虚拟线程名字中抽取平台线程名并进行消重处理，可以知道具体有哪些平台线程参与

可以看到，提交了20个工作任务，就创建了20个虚拟线程，但对应的平台线程，只有13个。